Regulatory & Support Copilot — Project Proposal

Oct 19, 2026

1. Project Overview

We propose a product that any organization can customize: a team of AI agents that watches new rules and answers incoming requests, while people approve every result.

The product is called the **Regulatory & Support Copilot**. The code is the same for every customer. Each organization fills in a **template** with its own words, departments, documents and records.

We build two templates to prove it works anywhere: a **university** (our main demo) and a **hospital**. Both use mock data that we create ourselves.

Note: all organizations and data are fictional. The project started from a case study of a Jordanian payments company and grew into a general product, as the doctor advised.

2. The Problem

Every organization spends a lot of staff time on two repeated, manual tasks.

1. **Handling new rules.** Authorities and management keep publishing new rules and policies (for example a new accreditation rule for a university, or a new health ministry rule for a hospital). Staff must read each one, find which departments and people it affects, and plan the actions.
2. **Answering requests.** Students, patients and customers ask questions like "Why can't I register for this course?" or "Why was my appointment cancelled?" Staff must dig through records to find the answer.

Why it happens

Cause	Result
Long rule documents, often in Arabic and English	Days of reading and summarizing
Many departments, programs and groups to check	Easy to miss who is affected
Actions planned by hand	Slow work and human mistakes

Cause	Result
Knowledge held by a few experienced staff	Delays when they are busy or leave
Answers hidden in records	Slow replies and frustrated students or patients

3. Objectives and Scope

The goal is to turn days of manual reading and searching into hours, for any organization, while staff stay in control.

Objectives

- Let an organization customize the system with a template file, without changing code.
- Summarize a new rule automatically: what changed and when it starts.
- Find which units (programs, departments) and which groups of people are affected.
- Draft action items with checks for staff to review.
- Find the cause behind a request in the records and draft a clear reply.

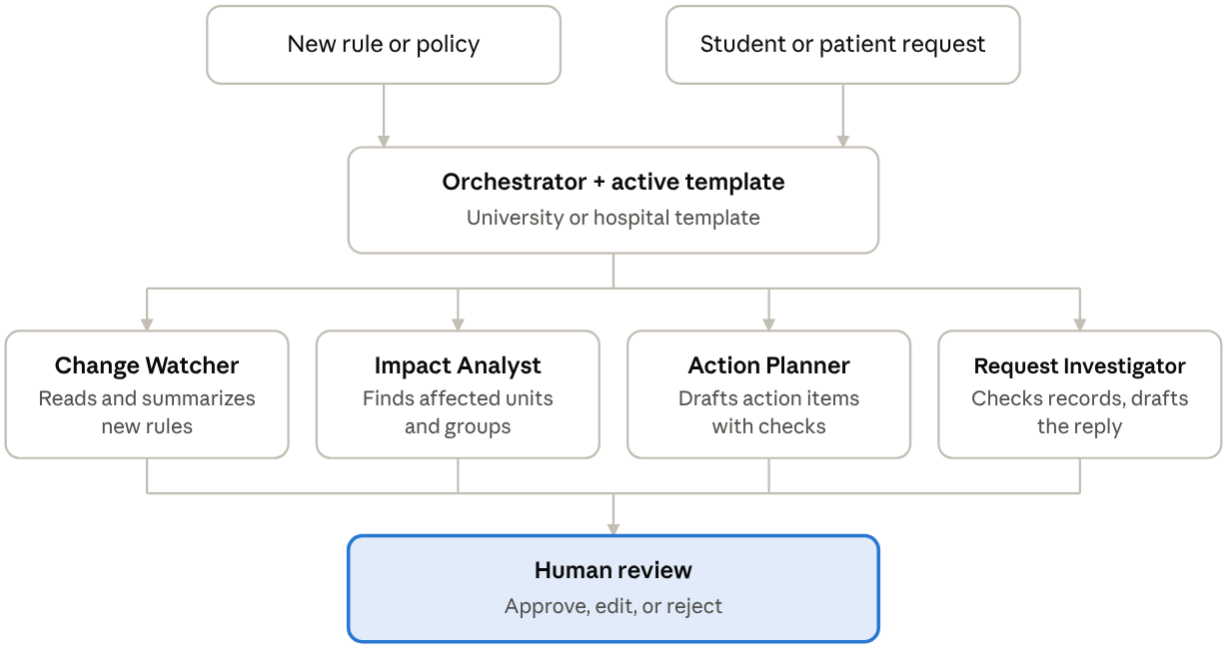
Out of scope

- The system does not change records or settings by itself.
- The system does not send anything to students or patients without human approval.
- No real organization data: only fictional mock data.

4. Proposed Solution

The solution is one core engine with a team of AI agents. A lead agent (the orchestrator) coordinates them, and the active template tells them which organization they work for.

Agents prepare the work, a human makes every decision



system design · 1 orchestrator, 4 agents, 1 human review

New rules go through **Change Watch**: the Change Watcher, the Impact Analyst and the Action Planner. Requests go through **Request Desk**: the Request Investigator. Switching the template from university to hospital changes the words, documents and records, not the code. All results stop at a human before anything is used.

5. How It Works

The system runs two workflows for whichever template is active. Each ends with a human review.

Change Watch: a new rule arrives (university example)

1. A new accreditation circular is uploaded.
2. The Change Watcher summarizes what changed and quotes the exact sentences.
3. The Impact Analyst lists the affected study programs and student groups.
4. The Action Planner drafts action items with checks.
5. The dean's office reviews and approves them.

Request Desk: a request arrives (university and hospital examples)

1. A student asks "Why can't I register for SE301?", or a patient asks why an appointment was cancelled.
2. The Request Investigator looks up the records through a read-only connector.

3. It finds the cause (a missing prerequisite, an unavailable doctor) and explains it in simple words.
4. A staff member reviews the reply and sends it.

Simple technical explanation

- **Template:** one file per organization with its labels, units, groups, documents and record connector.
- **Large Language Model (LLM):** the "brain" that reads, understands, and writes text.
- **RAG (Retrieval-Augmented Generation):** the system first searches the organization's own documents, then answers from them.
- **Vector database:** stores documents so they can be searched by meaning, one collection per template.
- **Connectors:** small plug-ins that read records (registrations for the university, appointments for the hospital).
- **Suggested stack:** Python, FastAPI, Chroma, SQLite, and a free local model (Ollama) or a cloud API.

6. Safety, Expected Outcomes, and Plan

Student and patient data is sensitive, so safety rules are built in from the start.

Safety rules

- A human approves every action item and every reply.
- Agents can read records but never change them.
- National IDs, phone numbers and medical record numbers are hidden before the AI sees them.
- Every action is saved in an audit log.

Expected outcomes

- A working prototype of both workflows, shown live on a demo website for two templates (university and hospital) with mock data.
- Faster rule handling and faster request replies, measured against manual work.
- Answers that point to the source document they came from.

Project plan

1. Design the template format and the shared data formats.
2. Create the university and hospital templates with mock documents, records and requests.

3. Build the document search (RAG) and the AI connection.
4. Build and test each agent.
5. Connect the agents to the demo website with the template switcher and review buttons.
6. Test, measure results for both templates, and present the live demo.